

Національний університет “Львівська політехніка”

Кафедра автоматизованих систем управління

Методичні вказівки до курсової роботи
з дисципліни “Прикладне програмування”

(автори – Ю.В. Цимбал, О.В. Скорохода; версія від 02.03.2024)

1. Основні цілі курсової роботи

- навчитись формулювати задачі для програмних проектів, розкладати їх на підзадачі, вибирати методи та алгоритми для їх розв’язання;
- навчитись визначати, порівнювати, вибирати і використовувати сучасні засоби та технології програмування для розв’язування конкретних прикладних задач;
- навчитись реалізовувати, відлагоджувати та розгортати програмні проекти з використанням сучасних технологій розробки;
- навчитись описувати програмні проекти, як кінцеві результати, так і процес виконання.

2. Завдання до курсової роботи

Створити програму на мові програмування Java згідно до індивідуального завдання.

1. Уважно пропрацюйте предметну область згідно вашого варіанту завдання. Опишіть технологічне завдання (який функціонал матиме ваша програма). Створіть **Use Case** та **Class** діаграми для вашого завдання.
2. Реалізуйте функціонал програми. Приклади індивідуальних завдань наведено у Додатку А. Саме завдання описане досить поверхово, якщо ви претендуєте на високу оцінку – його потрібно розширити. Наприклад, у завданні може бути сказано про «створити кілька квіток, з наявних квіток сформувати букет». Цілком логічно буде додати можливість видалити квітку, поміняти параметри існуючої квітки тощо.
3. Створити графічний інтерфейс користувача (з використанням будь-якої графічної бібліотеки, наприклад, JavaFX).

4. Нефункціональні вимоги:

- a. Покрийте вашу програму юніт тестами (coverage $\geq 90\%$).
- b. Додайте логгер з логуванням основних дій та виняткових ситуацій. При цьому далі логу мають зберігатися у файлі, а критичні помилки – додатково надсилатися на e-mail.
- c. Написати automation tests для графічного інтерфейсу (не обов'язковий пункт, для тих, хто претендує на оцінку "відмінно")

Окрім цього, програма повинна відповідати таким додатковим вимогам:

- Використовувати можливості ООП: класи, успадкування, поліморфізм, інкапсуляцію (успадкування потрібно використовувати тільки тоді, коли воно має сенс.)
- Кожний клас повинен мати назву, яка повністю описує його суть, і інформативний склад.
- При записі програми потрібно використовувати домовленості щодо оформлення коду java code convention.
- Класи потрібно грамотно розкласти по пакетах.
- Для збереження даних використати базу даних (SQL або NoSQL, на вибір студентів)

3. Вимоги до змісту та структури курсової роботи

Зміст **пояснювальної записки** до роботи може бути таким:

Титульна сторінка.

Завдання на курсову роботу.

Зміст.

Вступ.

1. Постановка задачі.
2. Огляд літератури.
3. Опис етапу проектування.
4. Програмне рішення.
5. Опис проведених експериментів.

Висновки.

Список використаних джерел.

Додатки.

Рекомендований обсяг пояснювальної записки – 20-25 сторінок (без додатків). Викладення матеріалу має бути збалансованим, розмір кожного розділу не має перевищувати 20% загального обсягу роботи.

Вимоги до оформлення роботи наступні:

1. Шрифт Times New Roman, розмір шрифту 14. Заголовки розділів – 16, bold. Кожен розділ починати з нової сторінки.
2. Міжрядковий інтервал – 1.5 лінії.
3. Абзац – 1.25 см.
4. Вирівнювання тексту – по ширині (Justify)
5. Зображення (рисунки) мають бути хорошої якості, з контрастним читабельним текстом на них і бажано світлим фоном, і підписані (наприклад, “Рис. 1. Діаграма випадків використання”). Великі зображення, наприклад, діаграму класів можна розбити на декілька менших, або розмістити у додатку на альбомній сторінці (Landscape).

4. Вимоги до виконання окремих розділів пояснювальної записки курсової роботи

У **вступі** слід описати мету, яка ставиться в роботі, її актуальність, можливості вирішення сучасними програмними засобами, майбутнє застосування результатів роботи на практиці. Вступ можна написати після завершення роботи.

Постановка задачі – це чітке визначення функціональних можливостей програмної розробки, вимоги до неї з боку користувача, зокрема, тип програмного рішення – настільний (desktop), веб-орієнтований або мобільний застосунок.

Огляд літератури розкриває особливості предметної області, для якої розробляється програмний проект, визначає місце проекту серед схожих за призначенням інших проектів, знаходить близькі прототипи та підходи до розв’язання аналогічних задач. Включає порівняльний аналіз можливих варіантів розв’язання поставленої задачі та вибір оптимального варіанту.

Мета етапу проектування – максимально спростити задачу, розбиваючи її на під задачі, розробити загальну архітектуру та окремі частини програмної системи. Для опису результатів можна використати UML-діаграми, зокрема, діаграма прецедентів (Use Case), діаграма компонентів, діаграми класів та схема бази даних, які описують будову програмної системи та її функціонування і формують об’єктну модель системи (Докладніше про діаграми - в окремому розділі цих методичних вказівок).

Програмне рішення – це докладний опис Вашої роботи над програмною реалізацією поставлених задач. Зокрема, звернути увагу на розробку графічного інтерфейсу користувача

і взаємодію з базою даних. Також зазначити, які бібліотеки (крім стандартних) та приклади програм, написані іншими програмістами, були використані в роботі. При цьому можливість такого використання має бути дозволена з погляду авторського права і погоджена з керівником роботи.

Експериментальна частина включає опис послідовності дій, потрібних для використання програмної розробки – з погляду користувача (як завантажити і використовувати), адміністратора (як розгорнути на новому комп'ютері або сервері), програміста (як продовжити роботу над проектом за іншим комп'ютером). Також навести невеликі екранограми (знімки екрану роботи) з прикладами виконання різноманітних практичних задач.

Список використаних джерел має містити не менше 10 позицій (у т.ч. іноземних та з мережі Інтернет) з обов'язковими посиланнями у тексті пояснювальної записки.

Висновки це підсумок проведеної роботи із узагальненим оглядом задачі, яка була вирішена, перспективами розвитку програмної системи, коротким описом набутих Вами знань та вмінь.

Додатки містять тексти програмних модулів, таблиці вхідних та вихідних даних та великі екранограми, які ілюструють інтерфейс користувача розробленої програми.

5. Захист роботи

Перед захистом роботи керівник розглядає програмну та описову частину роботи, зокрема, перевіряє їх оригінальність. Для цього попередньо розмістіть **програмний код** на відкритому для керівника і закритому для інших репозиторії, наприклад, GitHub, а **пояснювальну записку** - на сторінці курсової роботи у Віртуальному навчальному середовищі.

Критерії оцінювання роботи:

1. 50% – програмна частина;
2. 50% – описова частина.

Захист може бути разом із презентацією Вами роботи. Тоді критерії оцінювання є такими:

1. 40% – програмна частина;
2. 40% – описова частина.
3. 20% – доповідь із презентацією

Оцінка виставляється при особистому спілкуванні з автором. Тобто важливими є як виконання та письмове оформлення роботи (50 %), так і усна компонента (50 %). Можливо, що захист роботи буде відбуватися перед комісією з трьох викладачів кафедри.

Додаток А. Варіанти індивідуальних завдань

1. **Квіти.** Визначити ієрархію квітів. Створити кілька об'єктів-квітів. Зібрати букет (використовуючи аксесуари) з визначенням його вартості. Здійснити сортування квітів у букеті на основі рівня свіжості. Знайти квітку в букеті, що відповідає заданому діапазону довжин стебел.
2. **Новорічний подарунок.** Визначити ієрархію цукерок та інших солодощів. Створити кілька об'єктів-цукерок. Зібрати дитячий подарунок з визначенням його ваги. Здійснити сортування цукерок в подарунку на основі одного з параметрів. Знайти цукерку в подарунку, що відповідає заданому діапазону вмісту цукру.
3. **Домашні електроприлади.** Визначити ієрархію електроприладів. Включити деякі в розетку. Підрахувати споживану потужність. Здійснити сортування приладів у квартирі на основі потужності. Знайти прилад в квартирі, що відповідає заданому діапазону параметрів.
4. **Шеф-повар.** Визначити ієрархію овочів. Зробити салат. Підрахувати калорійність. Здійснити сортування овочів для салату на основі одного з параметрів. Знайти овочі в салаті, що відповідають заданому діапазону калорійності.
5. **Звукозапис.** Визначити ієрархію музичних композицій. Записати на диск збірку. Підрахувати тривалість. Здійснити перестановку композицій диску на основі належності до стилю. Знайти композицію, що відповідає заданому діапазону довжини треків.
6. **Камені.** Визначити ієрархію дорогоцінних і напівкоштовних каменів. Відібрати камені для намиста. Підрахувати загальну вагу (в каратах) і вартість. Провести сортування каменів намиста на основі цінності. Знайти камені в намисті, що відповідають заданому діапазону параметрів прозорості.
7. **Лицарі.** Визначити ієрархію амуніції лицаря. Екіпірувати лицаря. Підрахувати вартість. Здійснити сортування амуніції на основі ваги. Знайти елементи амуніції, що відповідають заданому діапазону параметрів ціни.
8. **Транспорт.** Визначити ієрархію рухомого складу залізничного транспорту. Створити пасажирський потяг. Підрахувати загальну кількість пасажирів та багажу. Здійснити сортування вагонів поїзда на основі рівня комфортності. Знайти вагони в поїзді, що відповідають заданому діапазону параметрів кількості пасажирів.
9. **Авіакомпанія.** Визначити ієрархію літаків. Створити авіакомпанію. Підрахувати загальну місткість і вантажопідйомність. Здійснити сортування літаків компанії за дальністю польоту. Знайти літак у компанії, що відповідає заданому діапазону параметрів споживання пального.

- 10. Таксопарк.** Визначити ієрархію легкових автомобілів. Створити таксопарк. Підрахувати вартість автопарку. Здійснити сортування автомобілів парку за витратою пального. Знайти автомобіль в компанії, що відповідає заданому діапазону параметрів швидкості.
- 11. Страхування.** Визначити ієрархію страхових зобов'язань. Зібрати із зобов'язань дериватив. Підрахувати вартість. Здійснити сортування зобов'язань у деривативі на основі зменшення рівня ризику. Знайти зобов'язання у деривативі, що відповідає заданому діапазону параметрів.
- 12. Мобільний зв'язок.** Визначити ієрархію тарифів мобільної компанії. Створити список тарифів компанії. Підрахувати загальну чисельність клієнтів. Здійснити сортування тарифів на основі розміру абонентської плати. Знайти тариф у компанії, що відповідає заданому діапазону параметрів.
- 13. Фургон кави.** Завантажити фургон певного об'єму вантажем на певну суму з різних видів кави, що може знаходитися у різних фізичних станах (зернова, мелена, розчинна в банках і пакетиках). Враховувати об'єм кави разом з упаковкою. Здійснити сортування товарів на основі співвідношення ціни та ваги. Знайти товар у фургоні, що відповідає заданому діапазону параметрів якості.
- 14. Ігрова кімната.** Підготувати ігрову кімнату для дітей різних вікових груп. Іграшок повинна бути фіксована кількість в межах виділеної суми грошей. Повинні зустрічатися іграшки споріднених груп, наприклад: маленькі, середні і великі машинки, ляльки, м'ячі, кубики. Здійснити сортування іграшок у кімнаті по одному з параметрів. Знайти іграшки у кімнаті, що відповідають заданому діапазону параметрів.
- 15. Податки.** Визначити множину і суму податкових виплат фізичної особи за рік із врахуванням доходів з основного і додаткового місць роботи, авторських винагород, продажу майна, отримання в подарунок грошових сум і майна, переказів з-за кордону, пільг на дітей і матеріальну допомогу. Здійснити сортування податків по сумі. Знайти для особи податки, що відповідають заданому діапазону параметрів.
- 16. Вклади.** Сформуванати набір пропозицій клієнту по вкладах різних банків для оптимального вибору. Враховувати можливість дострокового зняття кредиту і/або поповнення. Реалізувати пошук і сортування вкладів за певними параметрами.
- 17. Туристичні путівки.** Сформуванати набір пропозицій клієнту з вибору туристичної путівки різного типу (відпочинок, екскурсії, лікування, шопінг, круїз і т.д.) для оптимального вибору. Враховувати можливість вибору транспорту, харчування і кількості днів. Реалізувати вибір і сортування путівок за певними параметрами.
- 18. Кредити.** Сформуванати набір пропозицій клієнту по цільових кредитах різних банків для оптимального вибору. Враховувати можливість дострокового погашення кредиту і/або збільшення кредитної лінії. Реалізувати вибір і пошук кредиту за певними параметрами.

Додаток Б. Діаграми реалізації програмного забезпечення

Це діаграми, за допомогою яких описується архітектура певної програмної системи.

Діаграма класів

Діаграма класів (Class diagram) визначає склад класів об'єктів і їх зв'язків. Крім того, діаграма класів може включати коментарі та обмеження. Обмеження можуть неформально задаватися на природній мові. Діаграма задається зображенням, на якому класи позначаються поділеними на три частини прямокутниками, а зв'язки – лініями, що з'єднують прямокутники. Це відповідає візуальному зображенню понять і зв'язків між ними. Верхня частина прямокутника – обов'язкова, в ній записується назва класу. Друга і третя частини прямокутника визначають відповідно список атрибутів класу (полів) і операцій (методів), що можуть мати такі специфікатори доступу (рис. 1):

- *public* (загальний) позначає операцію або атрибут, доступ до яких здійснюється з будь-якої частини програми будь-яким об'єктом системи;
- *protected* (захищений) позначає операцію або атрибут, доступ до яких здійснюється об'єктами того класу, в якому вони оголошені, або об'єктами класів-нащадків,
- *private* (приватний) позначає операцію або атрибут, доступ до яких здійснюється тільки об'єктами того класу, в якому вони визначені.

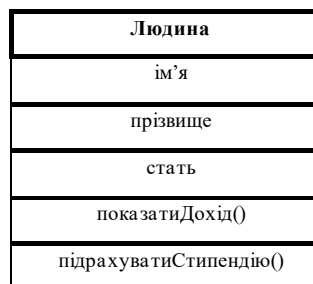


Рис. 1. Приклад наочного подання класу у UML

Користувач може визначати специфічні для нього атрибути. Під операцією розуміють сервіс, який екземпляр класу може виконувати, якщо до нього буде спрямовано відповідний виклик. Операція має назву і список аргументів. Для неї може також задаватися тип значення, яке вона повертає.

Класи можна побудувати з наступними відношеннями або зв'язками, які позначаються різними символами на діаграмі.

Асоціація – взаємна залежність між об'єктами різних класів, кожен з яких є її рівноправним учасником. Асоціації є механізмом, який надає об'єктам змогу обмінюватися даними між собою.

Асоціації можуть виконувати роль, яка визначає призначення асоціації і може бути одно- чи двосторонньою. Вона може позначати кількість екземплярів об'єктів кожного

класу, які беруть участь у зв'язку (0 – якщо жодного, 1 – якщо один, N – якщо багато).

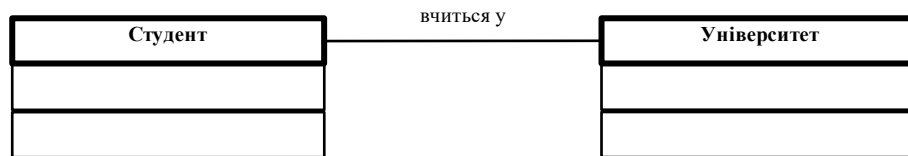


Рис. 2. Приклад асоціації у UML

У UML узагальнення між двома класами розташовує їх у вузлах ієрархії, яка відповідає концепції наслідування класу-нащадка від базового класу.

Наслідування є однією з фундаментальних рис об'єктно-орієнтованого програмування, коли клас “отримує” всі атрибути і операції класу, нащадком якого він є, і може перевизначати (змінювати) деякі з них, а також додавати власні атрибути і операції.

У UML узагальнення (наслідування) показують у вигляді лінії зі стрілочкою, що поєднує два класи і яку спрямовано до базового класу.

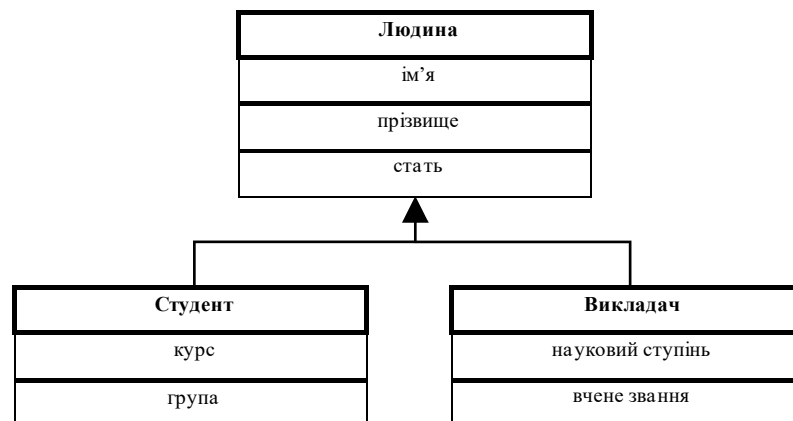


Рис. 3. Наслідування класів у UML

Залежність – відношення між класами, при якому клас-клієнт може використовувати певну операцію або сервіс іншого класу; класи можуть бути зв'язані відношеннями залежності, якщо один клас трансформується в інший внаслідок виконання певного процесу життєвого циклу програми.

Агрегація є особливим типом асоціації, за якого два класи, які беруть участь у зв'язку не є рівнозначними, вони мають зв'язок типу “ціле-частина” (приклад: “автомобіль-фара”).

Композиції – це асоціації, які відповідають дуже сильній агрегації. Це означає, що у композиціях також маємо справу з співвідношенням “ціле-частина”, але тут зв'язок є настільки сильним, що ціле не може існувати без частини (приклад: “автомобіль-двигун”).

Діаграма компонентів

Цей тип діаграм призначений для розподілення класів по компонентах під час фізичного проектування системи.

В UML – це діаграма, на якій відображаються компоненти, залежності та зв'язки між ними. Діаграма компонентів відображає залежності між компонентами програмного забезпечення, включаючи компоненти вихідних кодів, бінарні компоненти, та компоненти, що можуть виконуватися. На діаграмах компонентів зображуються входження класів і об'єктів у програмні компоненти системи (модулі, бібліотеки, бази даних і т.д.). Приклади діаграми наведено на рис. 4.

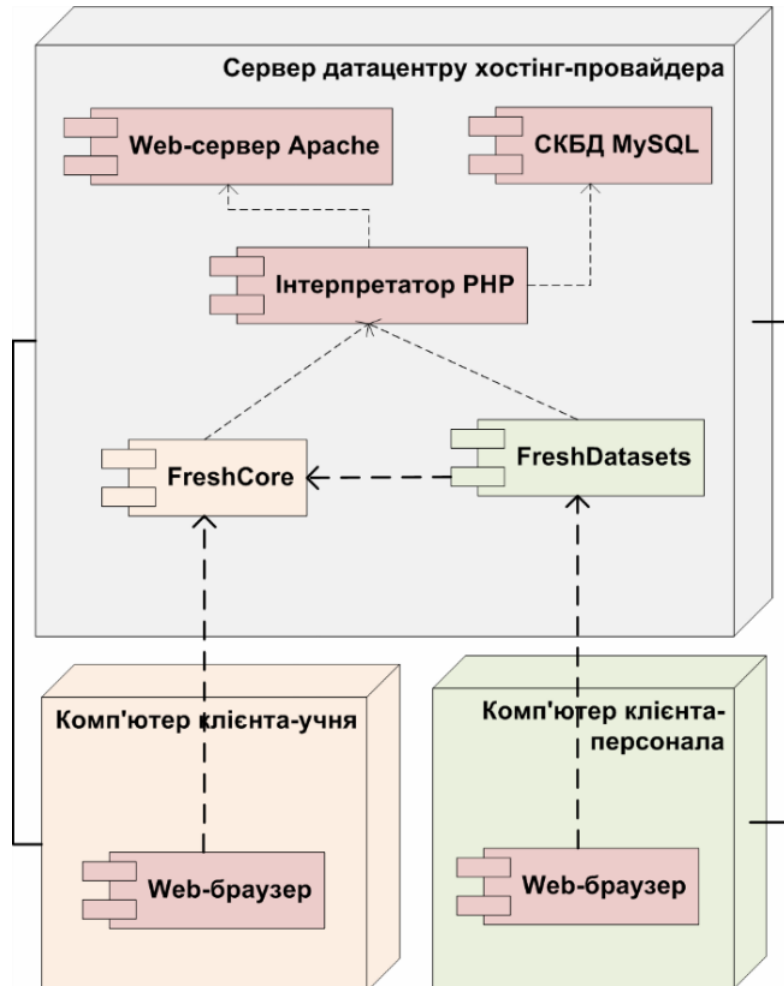


Рис. 4. Приклад діаграми компонентів